

SATA in two dimensions

with along-track and cross-track baselines

Implementation report: geometry, algorithm,
function reference and results

Lucas Ayub — DLR

September 2026

Summary

This is the 2-D version of SATA (Sub-Aperture Topography- and Aperture-dependent motion compensation), combined with the two-step Range-Doppler multichannel azimuth reconstruction, for a full 3-D acquisition geometry: along-track baselines b_{at} **and** cross-track baselines b_{xt} , with targets at arbitrary height. The cross-track baseline is what makes the problem non-trivial: with a purely along-track array the reconstruction is blind to topography (0.008° of phase per 100 m of range error), so there is nothing for SATA to correct.

On the geometry of the existing 1-D SATA tests, SATA recovers **99.0–100.4 %** of the ideal focused peak in all nine sweep cases, against 8.3–99.5 % without it. For topography that varies *along azimuth* — the case a single global correction cannot fix — the **per-sub-band** scheme reaches **91.3 %** of the monostatic reference, against 70.7 % for a whole-band correction and 59.1 % with none.

Contents

1	How to run it	2
1.1	IRF playground: <code>run_sata_irf.py</code> / <code>run_sata_irf_all.py</code>	2
2	Flowcharts	3
3	SATA 1-D vs. SATA 2-D	7
4	Geometry	9
4.1	Tracks and scene	9
4.2	Why a cross-track baseline is required	10
5	The algorithm	11
5.1	Doppler, angle and azimuth position	11
5.2	Multichannel reconstruction, in brief	11
5.3	The topographic residual	11
5.4	Where the correction is deposited	12
5.5	The SATA kernel	12
6	Function reference	13
7	Results	16
7.1	Single elevated target	16
7.2	Two-dimensional impulse response	17
7.3	Topography varying along azimuth	18
7.4	The same case at different ranges	19
8	Limitations and next steps	20

1 How to run it

The package needs only `numpy`, `scipy` and `matplotlib`; no proprietary library is imported and `h5py` is optional.

```
pip install numpy scipy matplotlib

# from the directory that CONTAINS sata2d/
python -m sata2d.run_sata2d_topo --plots
```

flag	effect
(none)	full run: self-test, residual table, 9-case sweep, azimuth topography (~4 min)
-quick	three sweep cases instead of nine (~1 min)
-skip-sweep	straight to the azimuth-topography experiment (~20s)
-plots	write the four figures
-outdir DIR	where the figures go (default <code>plots/sata2d</code>)

The script also runs directly: `python sata2d/run_sata2d_topo.py`.

1.1 IRF playground: `run_sata_irf.py` / `run_sata_irf_all.py`

Two smaller scripts are meant for quick, single-target experimentation rather than a full run: both build the same standard geometry as the rest of this report ($w_l=0.25$ m, $H=720$ km, $r_0=766.21$ km, $PRF=2000$ Hz, $N_{rx}=4$), place *one* target at a chosen azimuth offset and height, and reconstruct it. Only the `TARGET GEOMETRY` block at the top of each file is meant to be edited – everything else stays at the report’s defaults, so results across runs stay directly comparable.

```
python -m sata2d.run_sata_irf --method sub      # one method, one IRF
python -m sata2d.run_sata_irf_all              # all three, one plot
```

`run_sata_irf.py` reconstructs with a single method (`-method no | whole | sub`) and plots *ref* against *rec*; `run_sata_irf_all.py` reconstructs all three (no SATA / whole band / per sub-band) and overlays all four curves against the monostatic reference. Both plot the azimuth impulse response in dB over the *full* azimuth time axis, with a parameter box (N_{rx} , PRF , B_a , Δb_{at} , $b_{xt}^{\max}$) – the same look as `sar_recon.plotting.plot_combined`’s spectrum panel. Figure 1 shows `run_sata_irf_all.py`’s output for the default target (height 240 m, peak recovered to 8.0 % with no SATA vs. 95.4 % with SATA) and for a higher one (400 m, 58.8 % vs. 95.1 %). The no-SATA peak does not shrink monotonically with height – the residual is a *phase* error, so a bigger topographic offset can rotate it back closer to constructive interference at one particular target height, purely by chance – but SATA whole band and per sub-band stay around 95 % in either case, which is the point: the correction tracks the true residual geometrically rather than relying on it happening to fall favourably. Whole band and per sub-band coincide here because a single target has no azimuth-varying topography for the per-sub-band scheme to exploit (Section 3); that gain only shows up with several targets at different azimuths (Figure 11). The sidelobe spikes at ± 0.8 s and ± 1.6 s are the sub-aperture grating lobes, visible here only because the plot spans the full synthetic aperture rather than the usual ± 150 m window around the peak.

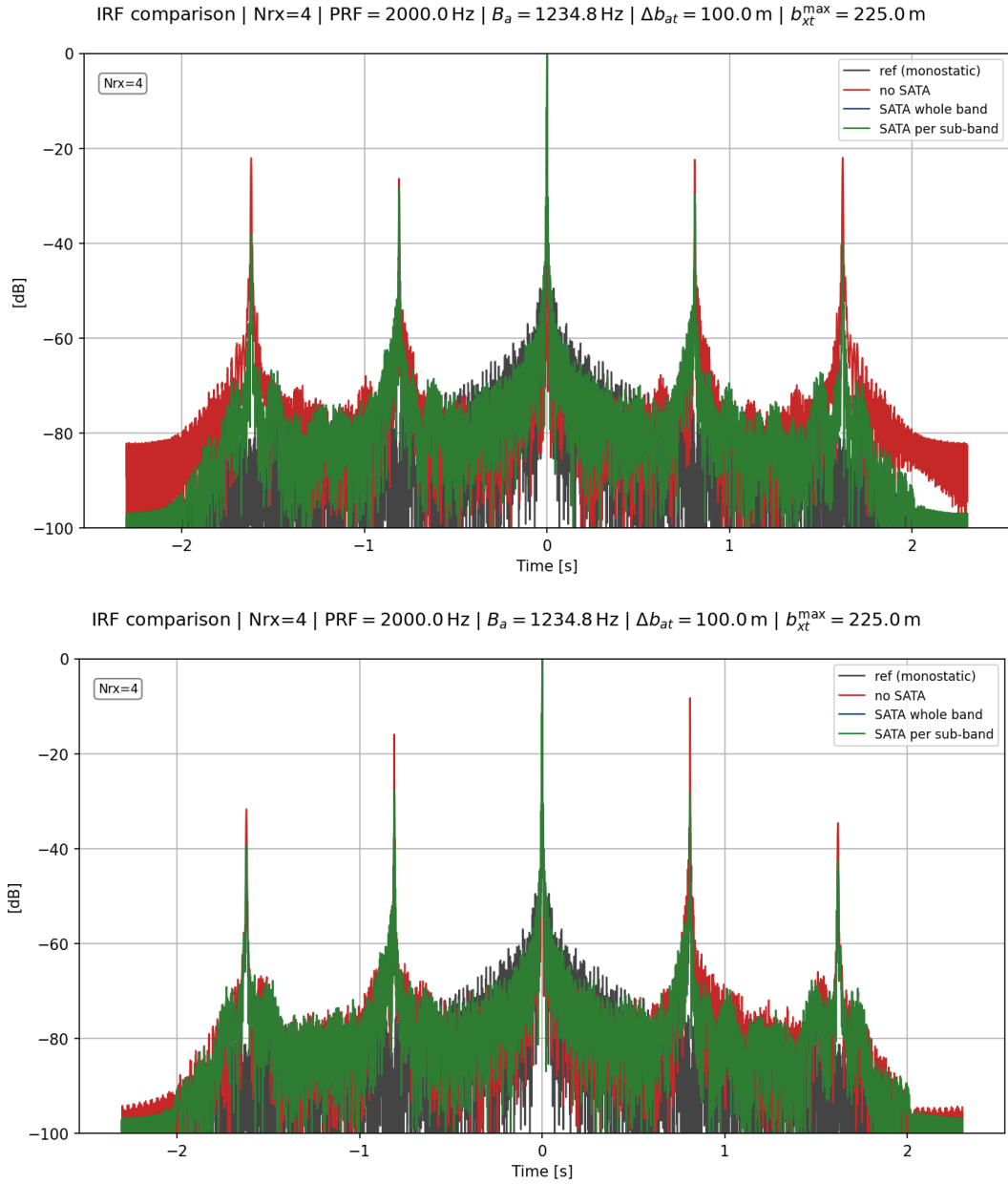


Figure 1: `run_sata_irf_all.py`, single target at azimuth 0, height 240 m (top) and 400 m (bottom): no SATA / SATA whole band / SATA per sub-band, against the monostatic reference. Whole-band and per-sub-band overlap almost exactly, as expected for a single target.

2 Flowcharts

Four diagrams, from coarse to fine. Figure 2 is the big picture: it answers “in what order do things happen, and where does SATA sit relative to the two-step reconstruction”. Figures 3 and 4 then zoom into the two blocks that actually do something to C_0 and to the data: the SATA kernel itself, and the two-step reconstruction that consumes whatever C_0 SATA (or the plain flat-Earth model) produced. Figure 5 finally zooms back out to show how the experiment script wires both of them together, sub-band by sub-band. Every block carries the file and function name, and the equation or the array shape it produces.

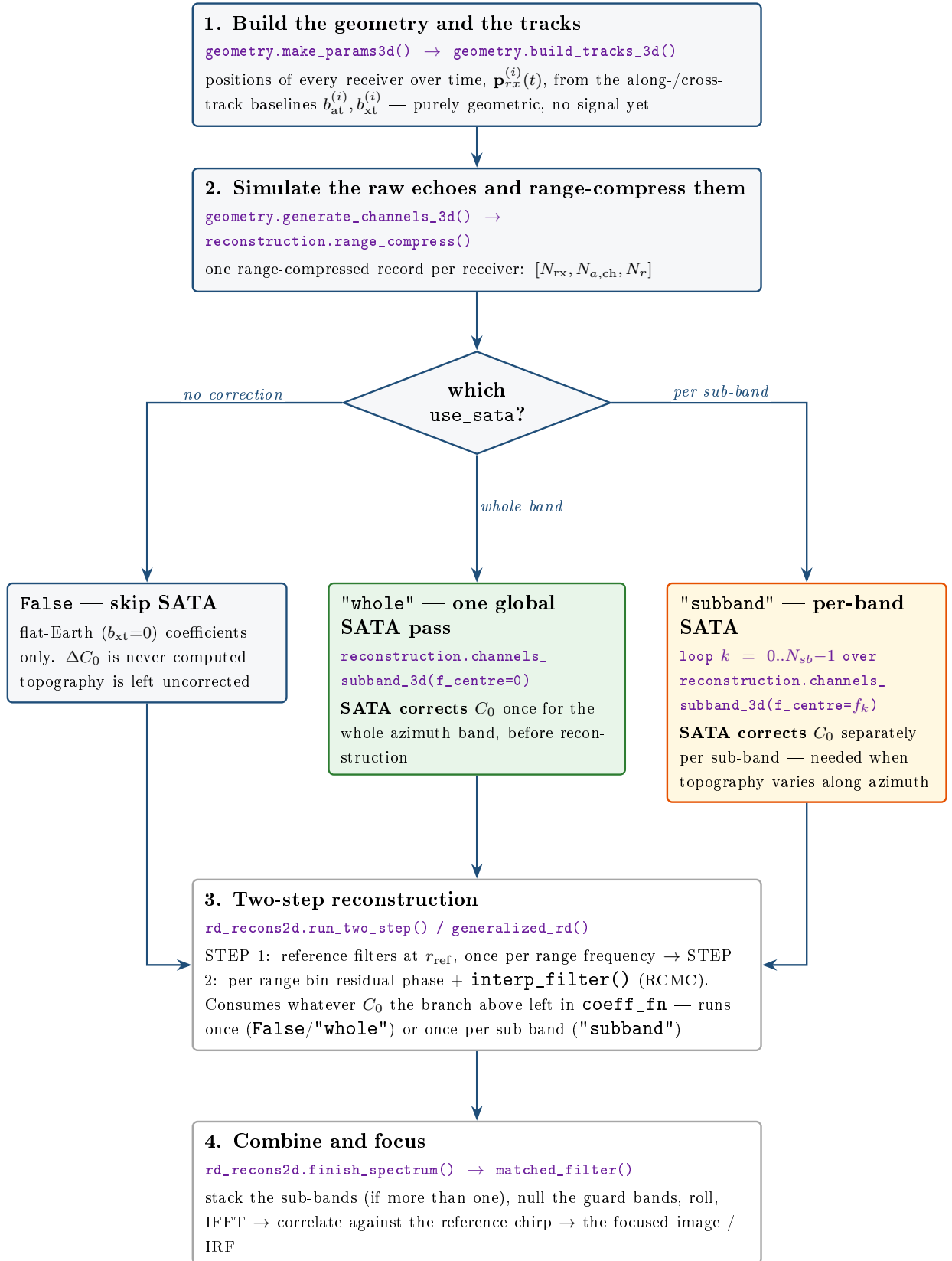


Figure 2: The big picture. Steps 1–2 (blue) only ever produce data and coefficients; SATA (green/orange, step in the middle row) is the block that actually touches C_0 , and it runs *before* the two-step reconstruction that consumes it — either once for everybody, once per sub-band, or not at all. Step 3 (grey) is the same two-step machinery from §3.2–3.3 in every branch; only the C_0 it is handed differs.

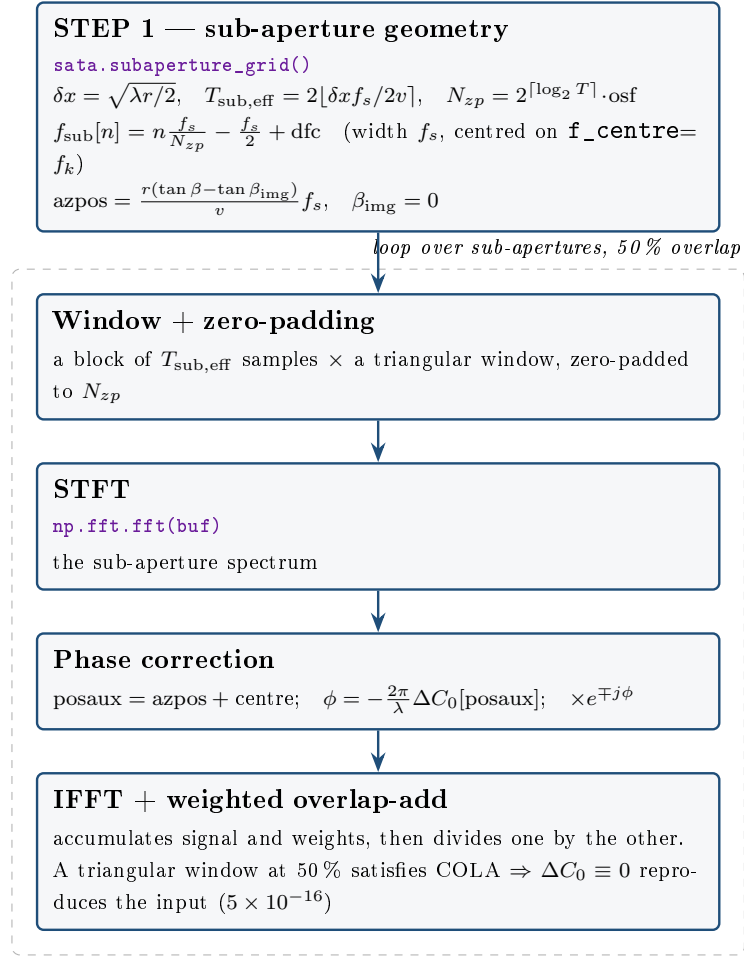


Figure 3: The SATA kernel — `sata.sata_1d()`. Everything that decides *where* on the ground a correction lands is in STEP 1.



Figure 4: The two-step reconstruction — `rd_recons2d.run_two_step()` / `generalized_rd()`. STEP 1 (top) fixes the *amplitude* of the channel-combination filter once, cheaply, at the reference range; STEP 2 (middle) only ever corrects the small *phase* left over at the target’s true range; RCMC (green) then slides that correction along range to follow true range-cell migration. This whole figure is the “STEP 1 / STEP 2” block of Figure 2 — *it is what the SATA branch in that figure feeds C_0 into*, via `coeff_fn`.

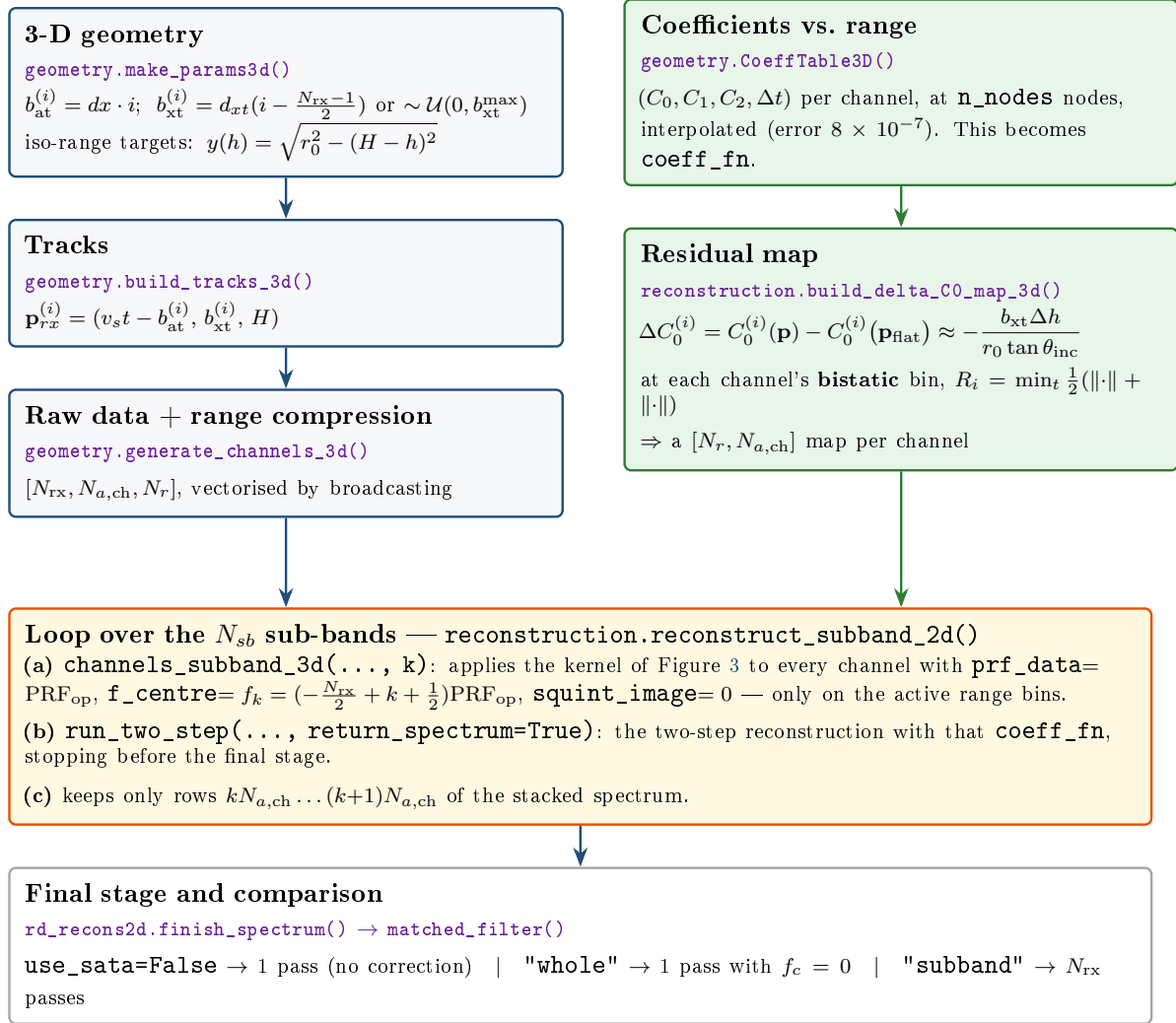


Figure 5: The experiment — `run_sata2d_topo.py`. The left branch produces the data; the right one (green) produces what the reconstruction filter does *not* know. The orange loop is the per-sub-band scheme.

3 SATA 1-D vs. SATA 2-D

The name is about *dimensionality of the correction domain*, not about the acquisition geometry. The existing per-sub-band SATA (`sar_recon/subband_recon.py`, `reconstruct_subband`, and its `"explicit"` variant) is, for reconstruction purposes, a **1-D** problem: it fixes a single reference range,

```

sceneMid = cfg.scene.ptg.reshape([3, 1])
Nr = 1
for mm in range(Nr):
    C0[kk], C1[kk], C2[kk], Dt[kk] = GetCoeffNu(sceneMid[:, mm], ...)
  
```

computes $(C_0, C_1, C_2, \Delta t)$ *once*, at that one range, builds the channel-combination matrix once, and reconstructs a single azimuth line per channel. There is no range-frequency loop, no per-range-bin residual, and no RCMC — there is nothing to migrate *along*, since there is only one range sample. It is exactly the right tool to validate the SATA frequency-to-azimuth mapping and the phase correction in isolation (Figure 3), but it never sweeps a range swath.

SATA 2-D (this package) resolves the same correction across a full range swath of N_r samples — genuinely **range** × **azimuth**: STEP 1 builds the reference filter once per range-frequency bin ($m = 0, \dots, N_r - 1$, Figure 4), STEP 2 recomputes $(C_0, C_1, C_2, \Delta t)$ at the true slant range of *every* range bin ($r_0 = r_{scan}[n]$, $n = 0, \dots, N_r - 1$, since these coefficients are not constant with range), and

RCMC (`interp_filter()`) slides the resulting residual filter along range to track how it migrates with azimuth frequency. None of that machinery is needed, or even meaningful, when $N_r = 1$. Figure 6 lays the two side by side; the reference files that motivated this package (the two-step range-Doppler reconstruction) are exactly what supplies that missing range dimension.

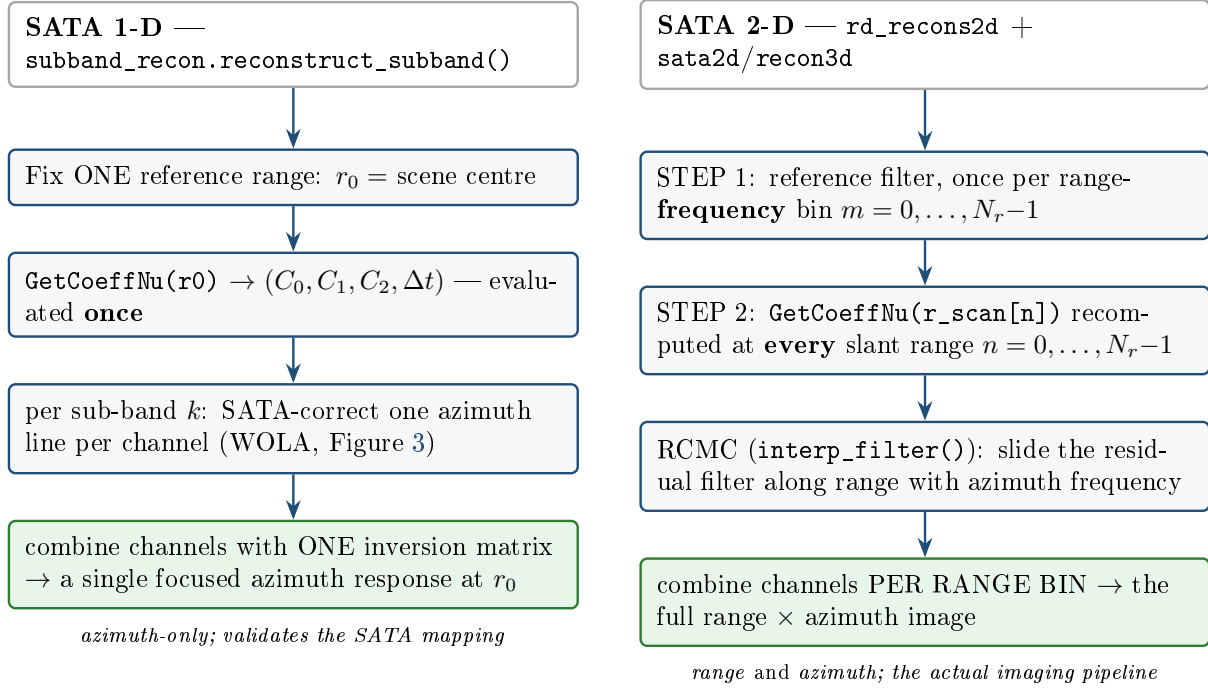


Figure 6: Same SATA kernel, two different processing domains. The left column has always existed and still exists, unchanged, inside the sub-band pipeline you already had; the right column is what this package adds — it is the left column’s phase correction, called once per range bin instead of once, inside the two-step reconstruction of Figure 4.

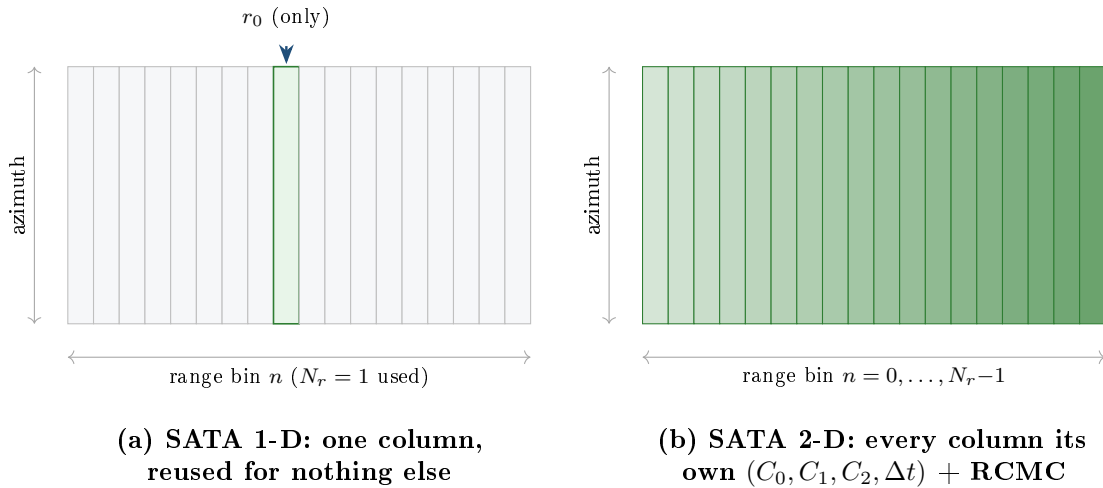


Figure 7: The same range $\times$ azimuth plane. (a) SATA 1-D only ever populates one column ($N_r = 1$): the coefficients are valid there and nowhere else, so nothing needs to migrate across range. (b) SATA 2-D fills every column with its own coefficients (shading is schematic, not an actual value), and RCMC is precisely the mechanism that keeps the azimuth correction registered to the right column as range changes.

4 Geometry

4.1 Tracks and scene

Coordinates $(x, y, z) = (\text{along-track}, \text{cross-track}, \text{height})$. Straight orbit:

$$\mathbf{p}_{tx}(t) = (v_s t, 0, H), \quad \mathbf{p}_{rx}^{(i)}(t) = (v_s t - b_{\text{at}}^{(i)}, b_{\text{xt}}^{(i)}, H). \quad (1)$$

Baselines:

$$b_{\text{at}}^{(i)} = \text{bat_offset} + dx \cdot i, \quad (2)$$

$$b_{\text{xt}}^{(i)} = \begin{cases} d_{xt} \left(i - \frac{N_{\text{rx}} - 1}{2} \right) & \text{"linear" mode,} \\ \sim \mathcal{U}(0, b_{\text{xt}}^{\text{max}}) & \text{"random" mode (seeded).} \end{cases} \quad (3)$$

The central target sits at $\mathbf{p} = (x_0, y_0, h_0)$ with $y_0 = \sqrt{r_0^2 - (H - h_0)^2}$; elevated targets are placed on the **iso-range** surface,

$$y(h) = \sqrt{r_0^2 - (H - h)^2} \implies \delta y = y(h_0 + \Delta h) - y(h_0) = \Delta h \cot \theta_{\text{inc}} + \mathcal{O}(\Delta h^2), \quad (4)$$

so they all land in the same range bin and any degradation is attributable to topography alone.

Default parameters reproduce the geometry of the existing 1-D SATA tests, so the 2-D results are directly comparable: $\lambda = 0.25$ m, $H = 720$ km, $r_0 = 766.21$ km ($\theta_{\text{inc}} = 20.0^\circ$), $d_a = 24\lambda$, $L_a = 2d_a$, $\text{PRF} = 2000$ Hz, $dx = 100$ m.

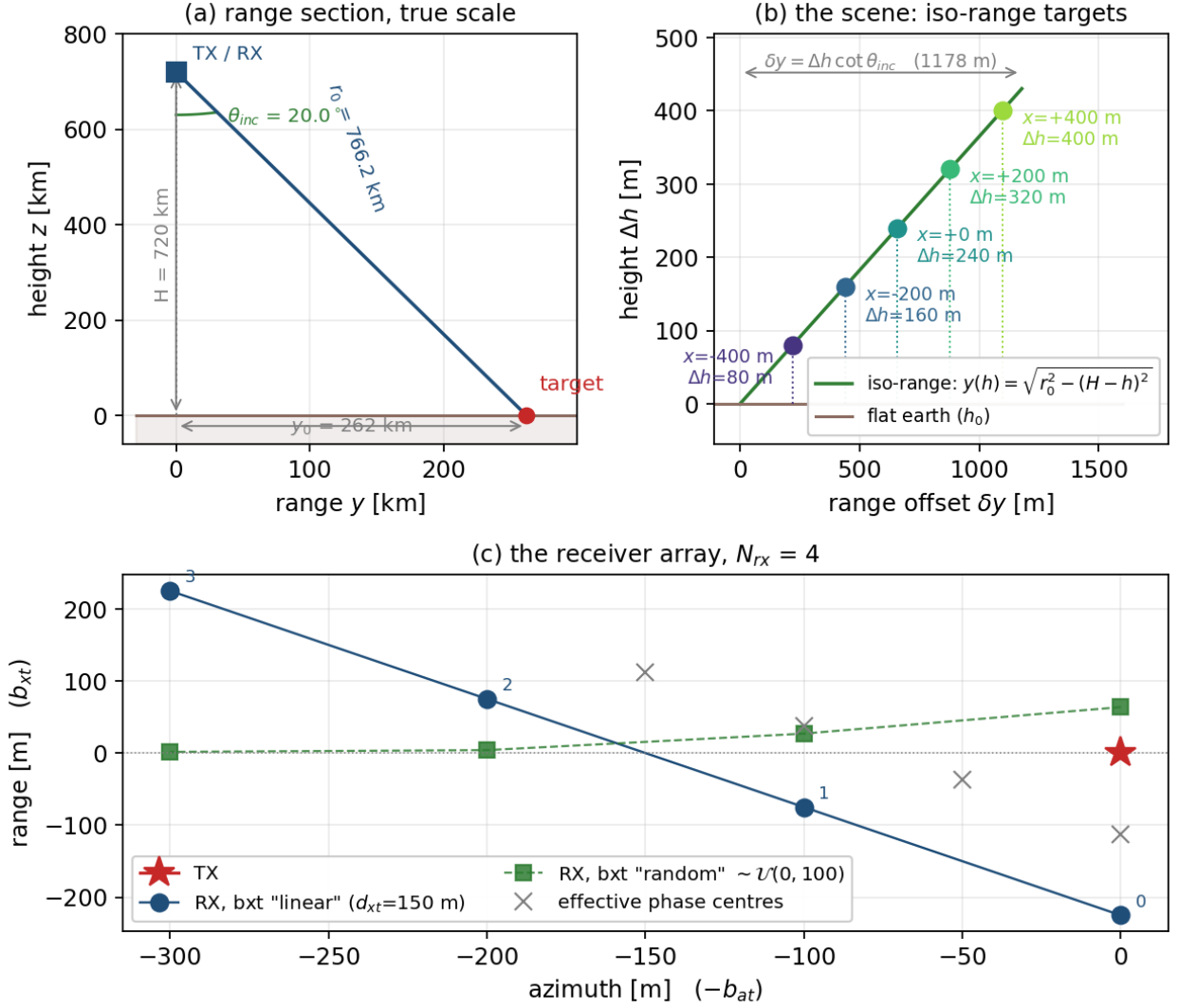


Figure 8: The geometry, read from the parameters (`run_sata2d_topo.plot_geometry()`). (a) range section at true scale. (b) the scene, zoomed: the five targets on the iso-range curve; rising by Δh slides the target in range by $\Delta h \cot \theta_{inc}$ (1178 m for $\Delta h = 400$ m), and it is that slide which turns the sin into a tan in Eq. (14). (c) the array in the (azimuth, range) plane, in both generation modes; the $\times$ are the effective phase centres, at $(-b_{at}/2, b_{xt}/2)$.

4.2 Why a cross-track baseline is required

For a purely along-track pair the constant term of the bistatic-minus-monostatic range history has the closed form $C_0 \approx b_{at}^2/4r$, which does *not* depend on target height. Differentiating,

$$\frac{\partial C_0}{\partial r} \approx -\frac{b_{at}^2}{4r^2} \implies \Delta C_0 = -\frac{b_{at}^2}{4r^2} \Delta r, \quad (5)$$

which for $b_{at} = 113$ m, $r = 700$ km and $\Delta r = 100$ m is 6.5×10^{-7} m — **0.008°** of phase. The reconstruction is effectively blind to topography and SATA has nothing to correct. All the sensitivity comes from b_{xt} .

5 The algorithm

5.1 Doppler, angle and azimuth position

For $|v_s t - x| \ll r$ the range history is a parabola in slow time, $r_h \approx 2r + (v_s t - x)^2/r$, giving

$$f(t) = -\frac{1}{\lambda} \frac{dr_h}{dt} = -\frac{2v_s(v_s t - x)}{\lambda r}, \quad k_a = \left| \frac{df}{dt} \right| = \frac{2v_s^2}{\lambda r}. \quad (6)$$

Inverting: a Doppler bin f sees the angle β and the azimuth position

$$\beta = \arcsin \frac{\lambda f}{2v_s}, \quad \boxed{\text{azpos}(f) = \frac{r \tan \beta}{v_s} f_s \approx \frac{\lambda r f f_s}{2v_s^2} = \frac{f}{k_a} f_s} \quad (7)$$

in samples of a line sampled at f_s . Everything in SATA follows from this line: it is what lets a correction that depends on *position* be applied in a domain indexed by *frequency*.

5.2 Multichannel reconstruction, in brief

Each channel samples at $\text{PRF}_{\text{op}} = \text{PRF}/N_{\text{rx}}$, so its spectrum is aliased. With $f_k = (-N_{\text{rx}}/2 + k + 1/2)\text{PRF}_{\text{op}}$ the centre of output sub-band k ,

$$U_i(f) = \sum_{k=0}^{N_{\text{rx}}-1} H_i(f + f_k) S(f + f_k) \iff \mathbf{u}(f) = \mathbf{H}(f) \mathbf{s}(f), \quad (8)$$

and `get_inversion_filters` solves $\mathbf{H}\mathbf{x}^{(k)} = N_{\text{rx}}\mathbf{e}_k$ per Doppler bin (the N_{rx} factor restores the amplitude lost to decimation). By stationary phase, with $r_h \approx a_0 + a_1 t + a_2 t^2$, the channel transfer function is

$$H_i(f) = \exp\left\{-2\pi j \left[\frac{C_0}{\lambda} + (-C_1 + \Delta t)f + C_2 \lambda f^2 \right]\right\}, \quad (9)$$

with

$$\begin{aligned} C_0 &= \delta c_0 + \frac{c_1^2}{4c_2} - \frac{(c_1 + \delta c_1)^2}{4(c_2 + \delta c_2)}, & C_1 &= \frac{c_2 \delta c_1 - c_1 \delta c_2}{2c_2(c_2 + \delta c_2)}, \\ C_2 &= \frac{\delta c_2}{4c_2(c_2 + \delta c_2)}, & \Delta t &= t_{bc}^{\text{bs}} - t_{bc}^{\text{ms}}, \end{aligned} \quad (10)$$

where c_k fits the monostatic history and δc_k the bistatic-minus-monostatic difference, both over the common aperture. `get_coeff_nu_3d` performs that fit on the actual 3-D tracks, which is what lets b_{xt} and target height enter.

The reconstruction is done in two steps because the filter depends on the slant range r_0 and on the range frequency f_r (through $\lambda(f_r) = c/(f_0 + f_r)$). STEP 1 builds it once, at the reference range, for every range frequency; STEP 2 applies, per range bin, only the phase difference

$$\tilde{P}_n(f) = e^{j \arg P(f; r_0)} \cdot \overline{e^{j \arg P(f; r_{\text{ref}})}}, \quad (11)$$

which is stable because the ill-conditioning of $\mathbf{H}$ was already handled in STEP 1.

5.3 The topographic residual

The filter is built at the flat-earth height h_0 . For a target at $h_0 + \Delta h$ the residual it fails to model is, exactly,

$$\boxed{\Delta C_0^{(i)}(\mathbf{p}) = C_0^{(i)}(\mathbf{p}) - C_0^{(i)}(\mathbf{p}_{\text{flat}}(r_{\mathbf{p}}))} \quad (12)$$

where $r_{\mathbf{p}} = \sqrt{y^2 + (H - h)^2}$ and $\mathbf{p}_{\text{flat}}(r)$ is the flat-earth point at the *same* slant range and azimuth — exactly what that range bin's filter assumes. Both terms come from the same fit, so correction and error come from one consistent model.

Closed form. A receiver displaced by b_{xt} sees the target at

$$r' = \sqrt{(y - b_{\text{xt}})^2 + (H - h)^2} \approx r - b_{\text{xt}} \sin \theta_{\text{inc}}, \quad (13)$$

so $C_0 \approx -b_{\text{xt}} \sin \theta_{\text{inc}}$ (plus the height-independent along-track term). On the iso-range surface $dy/dh = (H - h)/y = \cot \theta_{\text{inc}}$, hence $d(\sin \theta_{\text{inc}})/dh = \cos \theta_{\text{inc}}/(r \sin \theta_{\text{inc}})$ and

$$\Delta C_0 \approx -\frac{b_{\text{xt}} \Delta h}{r_0 \tan \theta_{\text{inc}}}. \quad (14)$$

Table 1: Exact residual (12) against the closed form (14), default geometry ($N_{\text{rx}} = 4$, $d_{xt} = 150$ m).

ch	b_{at} [m]	b_{xt} [m]	Δh [m]	exact [mm]	approx. [mm]	phase [°]
0	0	-225	100	80.63	80.68	116.1
3	300	+225	100	-80.64	-80.68	-116.1
0	0	-225	240	193.34	193.64	278.4
1	100	-75	240	64.45	64.55	92.8
0	0	-225	400	321.94	322.73	463.6
3	300	+225	400	-322.00	-322.73	-463.7

They agree to 0.25 %, so (14) is a real check rather than an order of magnitude (the code always uses the exact value). Note the size: up to **464°** of unmodelled phase. The usual interferometric expression $-b_{\perp} \Delta h / (r_0 \sin \theta_{\text{inc}})$ is for a target rising vertically with its *ground* position fixed; on the iso-range surface the extra $\cos \theta_{\text{inc}}$ appears and the sin becomes a tan.

5.4 Where the correction is deposited

Large b_{xt} moves the target between range bins

A receiver displaced by b_{xt} sees the target at a shorter range, so its half-path moves by $b_{\text{xt}} \sin \theta_{\text{inc}}/2$. For $b_{\text{xt}} = 450$ m and $\theta_{\text{inc}} = 20^\circ$ that is **77 m** — **about three range bins**. The residual must be deposited at each channel's **bistatic** bin,

$$R_i = \min_t \frac{1}{2} (\|\mathbf{p}_{tx}(t) - \mathbf{p}\| + \|\mathbf{p}_{rx}^{(i)}(t) - \mathbf{p}\|), \quad (15)$$

not at the monostatic one. Using the monostatic bin makes large-baseline cases fail *silently*: the $d_{xt} = 300$ m case then gives 43.9 % of the ideal peak instead of 99.9 %.

The second coordinate is the azimuth pixel, $N_{a,\text{ch}}/2 + (x - x_0)/(v_s/\text{PRF}_{\text{op}})$. The result is an $[N_r, N_{a,\text{ch}}]$ map per channel; bins with no topography stay at zero, so the (expensive) kernel only runs where it must.

5.5 The SATA kernel

The sub-aperture length comes from a resolution trade-off: a block of ground extent L has a position uncertainty of L (nothing distinguishes positions inside the block) and a spectral uncertainty of $\lambda r/2L$ (the resolution $\delta f = v_s/L$ propagated through (7)). Equating them,

$$\boxed{\delta x = \sqrt{\frac{\lambda r}{2}}} \implies T_{\text{sub,eff}} = 2 \left\lceil \frac{\delta x f_s}{2v_s} \right\rceil. \quad (16)$$

Each sub-aperture is windowed, zero-padded and transformed; every Doppler bin gets the phase

$$\phi(f) = -\frac{2\pi}{\lambda} \Delta C_0(x_p(f)), \quad x_p(f) = \text{sub-aperture centre} + \text{azpos}(f), \quad (17)$$

and the blocks are recombined by weighted overlap-add. A triangular window at 50 % overlap satisfies COLA, so $\Delta C_0 \equiv 0$ reproduces the input to machine precision.

Three unambiguous arguments instead of a (prf, Nsb) pair

A channel line has $N_{a,\text{ch}}$ samples at PRF_{op} and a spectrum PRF_{op} wide: it already *is* “one sub-band’s worth of samples”. From one sub-band to the next only the **frequency label** of each bin changes. The kernel therefore takes

- **prf_data** — the sampling rate of the array handed in (PRF_{op});
- **f_centre** — the **absolute** Doppler centre of the sub-band, f_k ;
- **squint_image** — the squint of the image grid ΔC_0 lives on. It is **0** here, because the reconstructed image is broadside and only the frequency *window* is squinted; setting it to β_k reproduces classic squint-processed SATA.

The last one matters more than it looks: subtracting $\tan \beta_k$ from the mapping when the image is broadside displaces the topography lookup by $r \tan \beta_k \text{PRF}_{\text{op}} / v_s = \pm 186.7$ samples (± 1006 m), with *opposite signs* in the sub-bands above and below zero Doppler. `legacy_sata_1d` keeps that behaviour available for comparison.

6 Function reference

The ten small library files below were later merged into three (`geometry.py`, `sata.py`, `reconstruction.py`), closer to the original three-file codebase — same functions, same code, fewer files, each merge verified numerically identical to the pre-merge version bit-for-bit. The tables below keep the original per-file grouping since it still reflects the logical split; each caption names the file the functions actually live in today, with the pre-merge file in parentheses.

geometry.py (was params3d.py)

<code>make_bxt(Nrx, dxt, mode, bxt_max, seed)</code>	Eq. (3): symmetric ladder ("linear") or $\mathcal{U}(0, b_{\text{xt}}^{\text{max}})$ with a seed ("random").
<code>iso_range_offset(r0, H, h0, dh)</code>	Eq. (4): the cross-track offset that keeps a target of height $h_0 + \Delta h$ at the same slant range.
<code>make_topo_ramp_azimuth(..., specs)</code>	A topography ramp along azimuth: each $(\text{dx}_{\text{az}}, \text{dh})$ becomes an iso-range scatterer. This is the case a single global correction cannot solve.
<code>Params3D</code>	The whole parameter set, with <code>PRF_op</code> , <code>fr</code> (range-frequency axis in raw FFT order), <code>idx_dec</code> , <code>theta_inc</code> , <code>points</code> and <code>flat_point_at_range(r)</code> — the flat-earth point a given range bin’s filter assumes.
<code>make_params3d(...)</code>	Assembles everything; the defaults are the geometry of Section 3.1.

geometry.py (was geom3d.py)

<code>build_tracks_3d(p)</code>	Eq. (1): <code>ptx</code> $[N_a, 3]$, <code>prx</code> $[N_{\text{rx}}, N_a, 3]$.
---------------------------------	---

<code>get_coeff_nu_3d(ptg, ptx, prx, ...)</code>	Eq. (10) from the actual tracks. Restricts to the beam window and the common instantaneous-Doppler band, aligns both histories on their points of closest approach, and fits the two polynomials. Returns <code>None</code> when the common aperture is too short, rather than a meaningless coefficient.
<code>residual_C0_3d(p, tracks, ptg, ch)</code>	Eq. (12), exact.
<code>dC0_approx(p, ch, dh)</code>	Eq. (14), with the derivation in the docstring.
<code>CoeffTable3D(p, tracks, n_nodes)</code>	$(C_0, C_1, C_2, \Delta t)$ versus range, per channel. One fit costs ~ 17 ms and the reconstruction would need $N_r \times N_{rx}$ of them; since the coefficients are smooth in r , the table evaluates them at <code>n_nodes</code> nodes and interpolates (measured relative error 8×10^{-7} , verifiable with <code>check()</code>). Called as <code>table(r0, channel)</code> , which is the <code>coeff_fn</code> signature.

geometry.py (was datagen3d.py)

<code>range_history / antenna_window</code>	$r_h = \ \mathbf{p}_{tx} - \mathbf{p}\ + \ \mathbf{p}_{rx} - \mathbf{p}\ $, and the two-way beam window $ \arcsin(\dot{r}/v) \leq \text{sq} + \theta/2$.
<code>echo_block(R, w, ...)</code>	The whole $[N_a, N_r]$ block by broadcasting fast time against the range history: ~ 0.6 s for 9216×128 . A rect window replaces the explicit start/end indices of a scalar implementation.
<code>generate_reference_3d</code>	Ideal monostatic signal at the full PRF — the comparison reference.
<code>generate_channels_3d</code>	Per-channel bistatic signals on the decimated azimuth axis ($[:: N_{rx}]$), $[N_{rx}, N_{a, \text{ch}}, N_r]$.

reconstruction.py (was rangecomp.py, siglib.py, datastore.py)

<code>range_matched_filter(fr, cd, rbw)</code>	$H(f) = \exp\{-j\pi f T_p(f/B_r - 1)\}$: the chirp matched filter plus the $T_p/2$ advance that puts the peak at the true delay.
<code>range_compress(data, ...)</code>	Applies it along the range axis, for any number of leading dimensions.
<code>fft2 / ifft2(x, nthreads, axis)</code>	Despite the name, <i>1-D</i> FFTs along axis; the thread argument is accepted and ignored.
<code>DataStore(filename, backend)</code>	<code>ds['CH'] [...]</code> over NumPy arrays in RAM or over an HDF5 file, so the same reconstruction code runs in-core or out-of-core.

reconstruction.py (was rd_recons2d.py)

<code>get_coeff_nu(dx, r, ...)</code>	The closed-form linear-orbit coefficients; the default when no <code>coeff_fn</code> is supplied.
<code>get_inversion_filters(Hf)</code>	Solves $\mathbf{H}\mathbf{x}^{(k)} = N_{rx}\mathbf{e}_k$ per Doppler bin; <code>analytic=True</code> uses the closed 2×2 inverse.

<code>create_ref_dataset(..., coeff_fn)</code>	STEP 1: the filter at r_{ref} for every range frequency. The coefficients are evaluated once; only the explicit λ in the phase varies.
<code>interp_filter</code>	Shifts the residual filter along range by $\Delta t_{\text{RCM}} f_s$ with $\Delta t_{\text{RCM}} = \frac{2(r-r_{\text{ref}})}{c} (1 - [1 - (\lambda f/2v)^2]^{-1/2})$.
<code>generalized_rd(...)</code>	STEP 2: Eq. (11) per range bin, sub-band sum, band nulling and the azimuth IFFT. <code>return_spectrum=True</code> stops before the last three stages so a caller can assemble the output sub-band by sub-band.
<code>run_two_step / finish_spectrum</code>	Allocation plus the shared final stage.

sata.py (was sata2d.py)

<code>subaperture_grid(rref, prf_data, v, wl, r, f_centre, squint_image)</code>	STEP 1: Eq. (16) for $T_{\text{sub,eff}}$, bins of width <code>prf_data</code> centred on <code>f_centre</code> , and the mapping (7) measured against the image grid <code>squint_image</code> . Returned as a <code>SubapertureGrid</code> object with a <code>report()</code> method, so the grid can be printed and asserted on.
<code>sata_1d(...)</code>	The kernel of Figure 3, applying (17). <code>inverse=True</code> <i>removes</i> the residual.
<code>legacy_subaperture_grid / legacy_sata_1d</code>	The earlier grid and kernel, kept for A/B comparison.
<code>subband_centre_frequency(k)</code>	$f_k = (-N_{\text{rx}}/2 + k + 1/2)\text{PRF}_{\text{op}}$.

reconstruction.py (was recon3d.py)

<code>scatterer_range_bistatic(p, tracks, ptg, ch)</code>	Eq. (15): where <i>that channel's</i> energy actually lands.
<code>build_delta_C0_map_3d(p, tracks, ch)</code>	The $[N_r, N_{a,\text{ch}}]$ map: each scatterer deposits (12) at its bistatic bin and azimuth pixel, interpolated across azimuth within a bin. <code>range_halfwidth</code> covers the residual migration ($\approx (v_s T_{\text{int}}/2)^2/2r_0 = 45\text{ m}$, about two bins).
<code>channels_subband_3d(..., k)</code>	Conditions every channel for sub-band k (<code>f_centre= f_k</code> , <code>squint_image= 0</code>), on the active range bins only.
<code>reconstruct_subband_2d(..., use_sata)</code>	<code>False</code> : plain reconstruction. <code>"whole"</code> : one whole-band SATA pass ($f_c = 0$) and one reconstruction. <code>"subband"</code> : one pass <i>and</i> one reconstruction per sub-band, keeping only that sub-band's rows — $N_{\text{rx}} \times$ the cost.

run_sata2d_topo.py

<code>test1_kernel</code>	$\Delta C_0 \equiv 0$ must reproduce the input; a constant ΔC_0 must apply exactly $-2\pi\Delta C_0/\lambda$.
<code>test2_residual</code>	The exact-vs-closed-form table of Section 3.3.

<code>_run_case / test3_sweep</code>	One elevated iso-range target, reconstructed four ways: no SATA / whole-band SATA / per-sub-band SATA / <i>ideal</i> (<code>CoeffTable3D(height=dh)</code> , i.e. the filter told the true height).
<code>test4_azimuth_topo</code>	Five iso-range targets at different azimuths and heights, focused with a <i>single</i> -target reference — otherwise the matched filter returns the autocorrelation of the scene, with spurious peaks at every target-to-target lag.
<code>plot_geometry / make_plots</code>	The four figures.

7 Results

7.1 Single elevated target

Focused peak as a percentage of the *ideal* reconstruction. $\max|\Delta C_0|$ is the worst per-channel residual.

case	$\max \Delta C_0 $ [°]	no SATA	SATA whole band	SATA sub-band
$d_{xt} = 50, \Delta h = 240$	93	48.1 %	99.8 %	99.8 %
$d_{xt} = 150, \Delta h = 240$	278	8.3 %	99.3 %	99.3 %
$d_{xt} = 300, \Delta h = 240$	557	95.8 %	99.9 %	99.9 %
$d_{xt} = 150, \Delta h = 400$	464	61.2 %	99.0 %	99.0 %
$b_{xt} \sim \mathcal{U}(0, 100)$	79	86.6 %	100.4 %	100.4 %
$b_{xt} \sim \mathcal{U}(0, 20)$	16	99.5 %	100.0 %	100.0 %
$N_{rx} = 2, d_{xt} = 150$	93	8.8 %	99.3 %	99.3 %
$N_{rx} = 6, d_{xt} = 150$	464	10.9 %	99.3 %	99.3 %
DPCA $dx = 11$	278	37.4 %	99.2 %	99.2 %

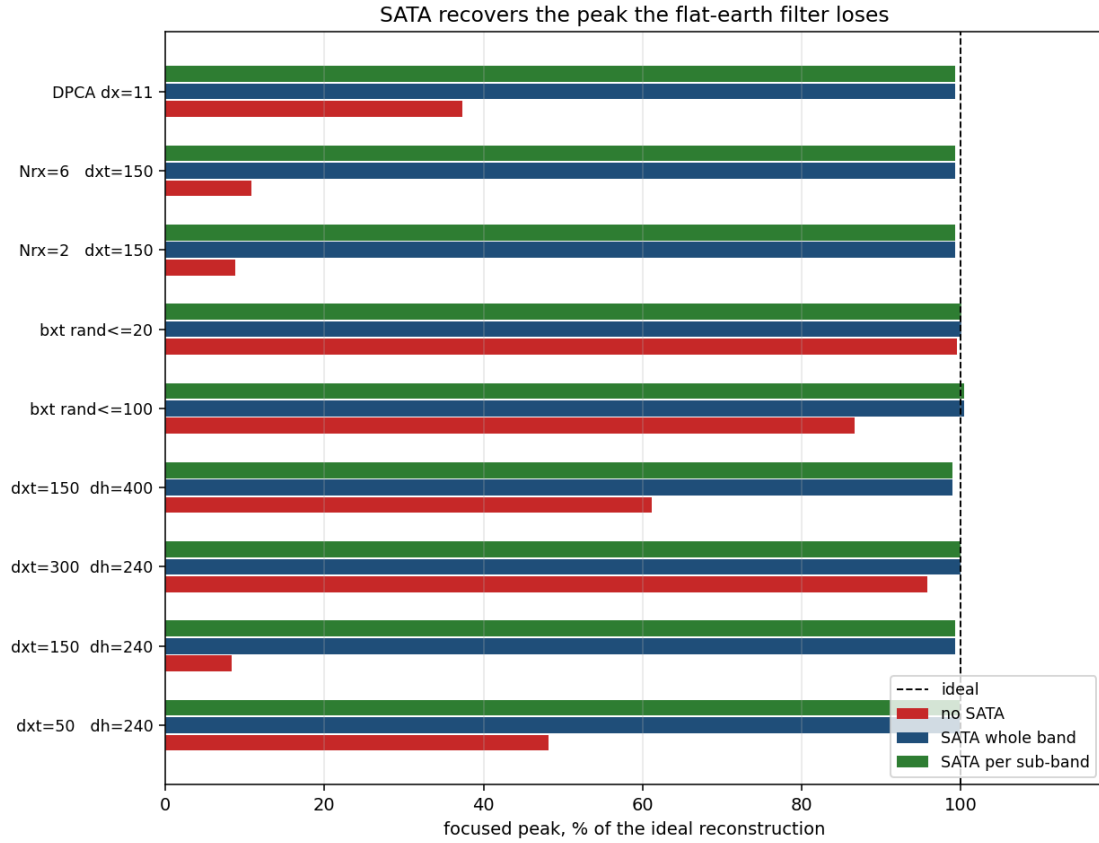


Figure 9: SATA recovers the peak the flat-earth filter loses — between 99.0 and 100.4% of the ideal in every case.

Why whole band = sub-band here

With a single target the residual is constant across azimuth, and C_0 is evaluated at closest approach — hence essentially angle-invariant. The two versions coincide, as they should. The per-sub-band machinery only earns its cost when the residual *varies* with position.

7.2 Two-dimensional impulse response

The IRFs above are azimuth cuts. `run_irf2d.py` produces the full two-dimensional response — a range $\times$ azimuth contour with the two 1-D cuts, in the style of Fig. 2.9 of Sakar’s dissertation. Two things are needed for that figure to mean anything.

First, **2-D matched-filter focusing**: $\text{img} = \mathcal{F}_2^{-1}[\mathcal{F}_2(x) \mathcal{F}_2(\text{ref})^*]$, with `ref` the ideal monostatic signal of the same target. This is exact, because the range cell migration — 41.6 m, eight range cells here — is contained in `ref` and is therefore compensated without any approximate RCMC. Focusing each range bin against the azimuth chirp of a single bin, the obvious shortcut, leaves that migration uncorrected and produces visibly skewed contours.

Second, **a range resolution comparable to the azimuth one**. The standard preset has $\delta_r = 30$ m against $L_a/2 = 6$ m in azimuth, a 5:1 ratio that flattens the cross into a bar. The script therefore defaults to `-res-rg 6`, matching the two; `-swath 300` keeps $N_r = 128$, i.e. the same cost as the standard preset.

As a check on the focusing, the monostatic reference returns PSLR -13.34 dB in azimuth and -13.47 dB in range against the theoretical rectangular-window value of -13.26 dB, with IRW(-3 dB) of 3.70 and 4.18 m for a 6 m resolution.

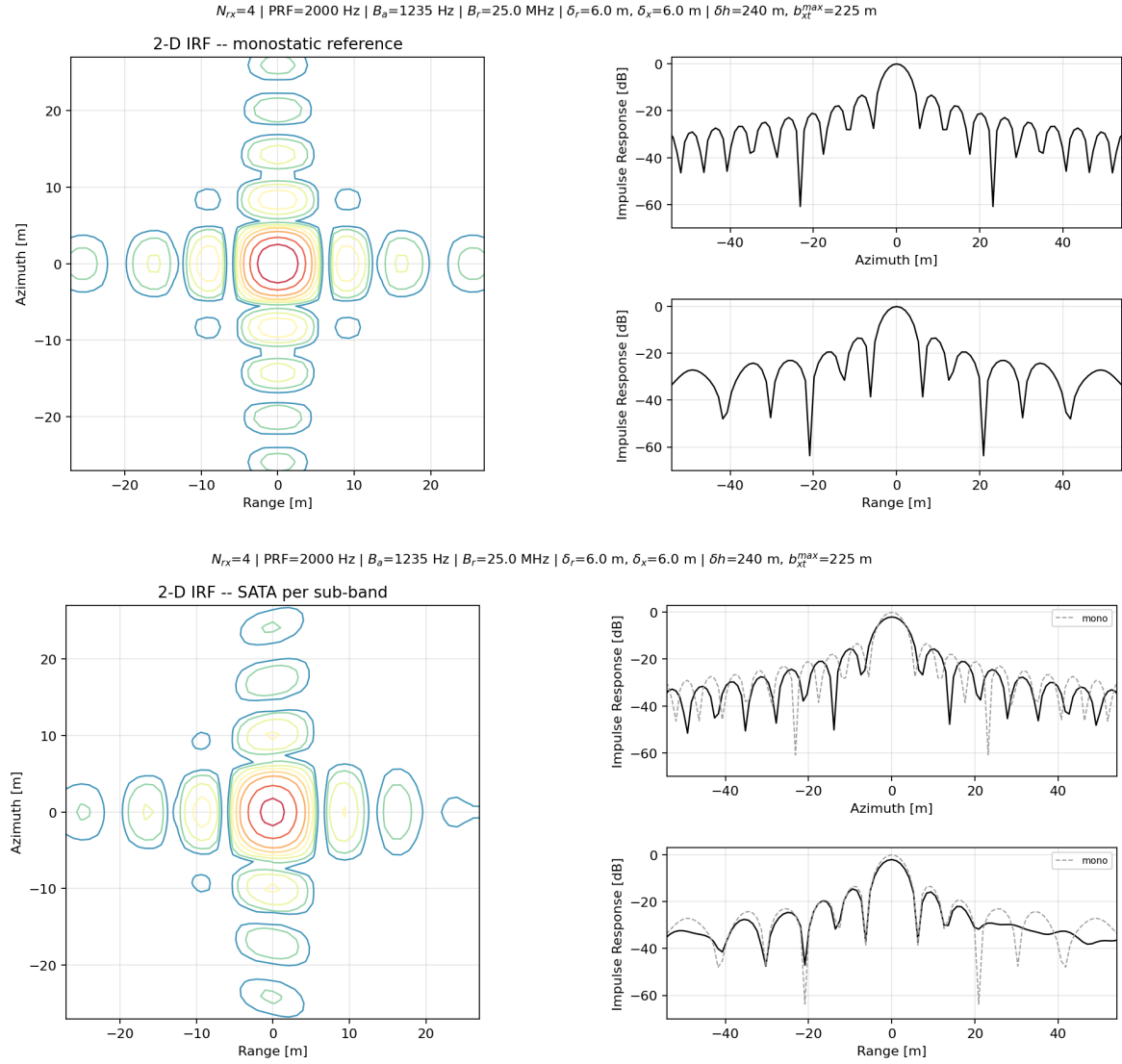


Figure 10: 2-D impulse response of the elevated point target, $\delta_r = \delta_x = 6$ m. Top: the monostatic reference, a clean 2-D sinc cross. Bottom: SATA per sub-band, with the monostatic cuts overlaid dashed. Left panel, the range $\times$ azimuth contour at -3 to -30 dB; right, the azimuth (top) and range (bottom) cuts. Produced by `run_irf2d.py -method mono` and `-method sub`.

7.3 Topography varying along azimuth

Five iso-range targets from -400 to $+400$ m in azimuth, heights $80 \dots 400$ m. Peak of the central target, as a percentage of the monostatic reference:

	peak	% of monostatic
monostatic reference	1.308×10^4	100.0 %
no SATA	7.725×10^3	59.1 %
SATA whole band	9.238×10^3	70.7 %
SATA per sub-band	1.194×10^4	91.3 %

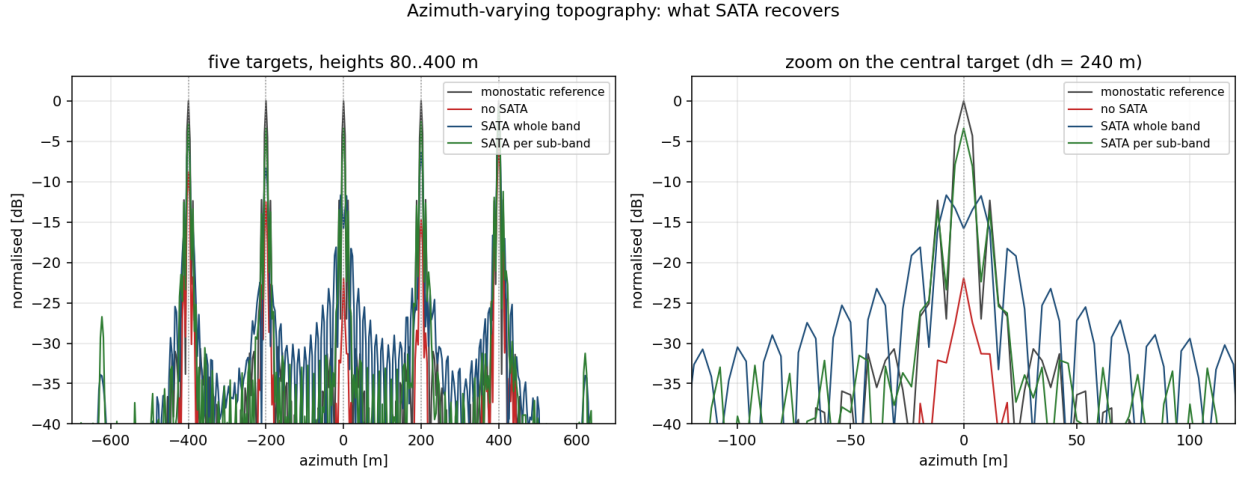


Figure 11: Five targets, heights 80...400 m. Green (per sub-band) tracks the monostatic reference; blue (whole band) recovers the peak but leaves elevated side lobes, because it can only apply *one* value per azimuth line; red (no SATA) loses -4.6 dB on the central target.

7.4 The same case at different ranges

Figure 11 above uses the one reference range every other result in this report uses ($\theta_{\text{inc}} = 20^\circ$, $r_0 = 766$ km). Since Section 3 makes a point of STEP 2 recomputing ($C_0, C_1, C_2, \Delta t$) at every range bin — not just that one — it is worth showing that this is not a one-range coincidence: the central iso-range target of the same five-target scene, reconstructed at $\theta_{\text{inc}} = 12^\circ$ (near range, $r_0 = 736$ km), 20° (mid, 766 km) and 28° (far, 815 km), everything else in the geometry left untouched.

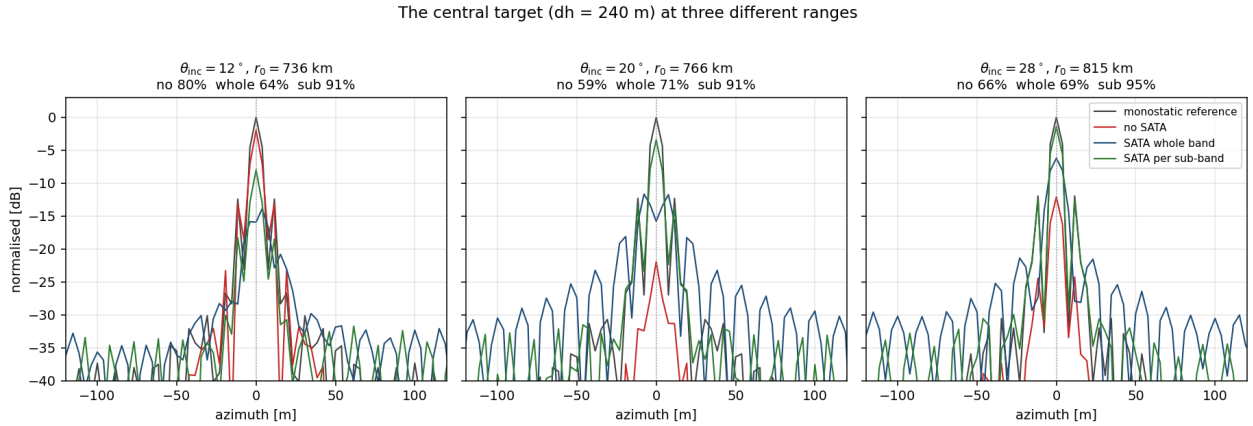


Figure 12: Same scene, same reconstruction, three different reference ranges. The per-sub-band correction (green) stays at 91–95 % of the monostatic peak throughout; a smaller incidence angle also means a smaller $b_{\text{xt}} \sin \theta_{\text{inc}}$ bistatic-bin shift and a shorter optimum sub-aperture $\delta x = \sqrt{\lambda r/2}$, which is why the 12° panel is the best-focused of the three even without SATA. Whole-band SATA is the one case worth a second look: at 12° it is *worse* than no correction at all (64 % vs. 80 %) — a single global phase, applied where the true residual varies most steeply with azimuth, can overcorrect as easily as it can undercorrect.

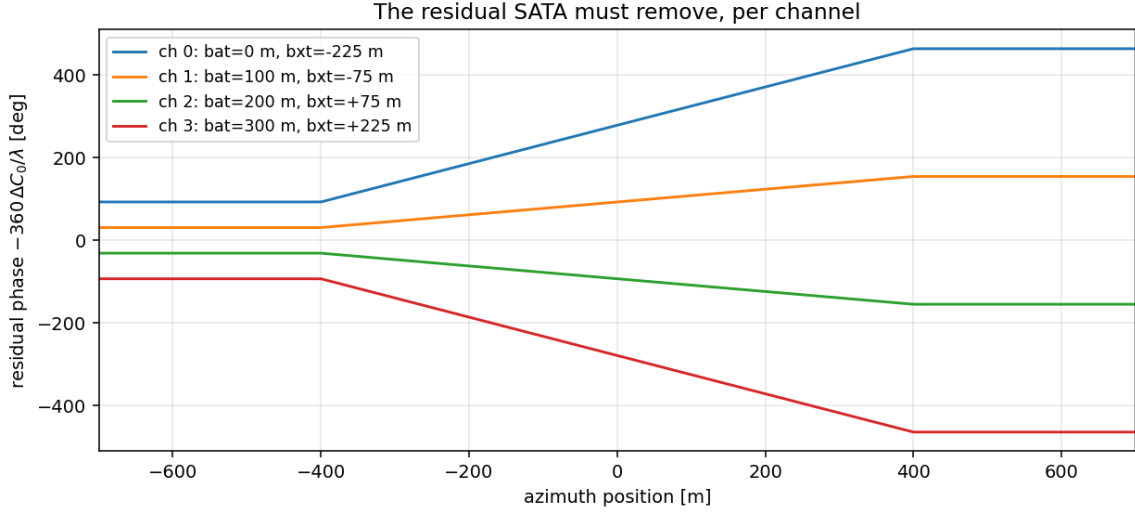


Figure 13: The residual SATA must remove, per channel, along azimuth. The slope is the topography ramp; sign and magnitude follow $b_{xt}^{(i)}$, as Eq. (14) requires. Between the outer channels there is more than 900° of difference.

The central result

This is where the **per-sub-band** scheme pays for itself: each sub-band has a different frequency window, which by (7) sees a different piece of the ground, so the correction becomes *position-selective*. 91.3 % against 70.7 % for the whole band and 59.1 % with no correction.

8 Limitations and next steps

1. **Only the C_0 term is corrected.** In broadside that is justified, and now measured: with δC_k the residual between the true target and the flat point at the same slant range, converted to phase over T_{int} , $|\varphi_0|_{\max} = 278.5^\circ$ while $|\varphi_1|/|\varphi_0| = 2.7 \times 10^{-8}$ and $|\varphi_2|/|\varphi_0| = 2.4 \times 10^{-9}$ — polyfit noise. Both $r_{ms}(t)$ and $r_{bs}(t)$ are even about their own points of closest approach, so the difference carries no odd term and $C_1 \equiv 0$; raising the target preserves that symmetry. An oracle reconstruction, given the true height so that C_0 , C_1 and C_2 are all exact, gains 0.7 percentage points over SATA with C_0 alone (96.1% against 95.4%) and nothing in error-to-signal. This stops holding under squint; `get_coeff_nu_3d` already returns C_1 and C_2 per sub-band, so the extension reuses the same machinery. Measured by `run_c1c2.py`.
2. **Inter-channel range co-registration is handled, but implicitly.** A cross-track baseline shifts each channel's energy by $b_{xt} \sin \theta_{inc}/2$ — 38.5 m, or 1.47 range cells, for $b_{xt} = 225$ m. That shift is *not* missing from the pipeline: the STEP 1 filter is built at every range frequency and its $C_0/\lambda(f_r)$ term expands into a carrier phase plus a range-frequency ramp, which by the shift theorem is exactly the co-registration. Removing the range-frequency dependence, and leaving the shift uncorrected, costs the focused peak 96.1% $\rightarrow$ 37.4%; an explicit co-registration on top of a monochromatic filter restores it to 96.1%, and applying both double counts (Figure 14). The practical consequence is that the `wl_arr` loop of `create_ref_dataset` is load-bearing and must not be simplified. See the companion report `docs/coreg_report_en.pdf` and `coreg.py`.
3. `apply_rcm` in STEP 1 is off by default, and `interp_filter` requires `Nb == Nr`.
4. `CoeffTable3D` interpolates in range; for very wide swaths increase `n_nodes` and verify with `check()`.

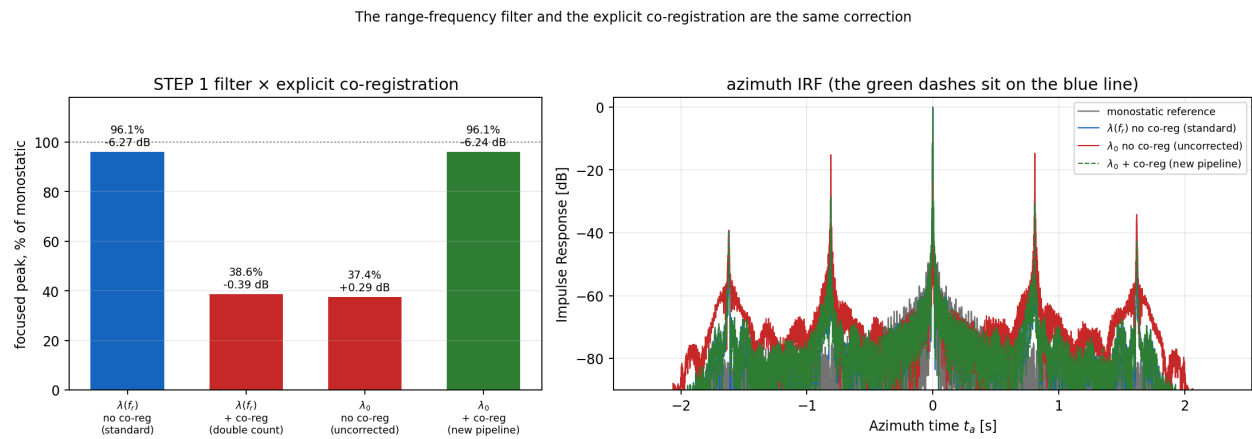


Figure 14: The range-frequency dependence of the STEP 1 filter and an explicit range co-registration are the same correction, and exactly one of them must be applied. Left, the four combinations; right, the resulting azimuth IRFs — without the correction the residual ambiguities rise from -28 to -15 dB. Produced by `run_coreg.py` and `plot_coreg.py`.